Reviewer Pack — Antichain-Width of Query Certificate Poset Bounds Lifted IND…

Entry 52be4733a1c4 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-04-27 14:50:35 UTC

Antichain-Width of Query Certificate Poset Bounds Lifted IND CC

Entry ID: `52be4733a1c4` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-04-27 14:50:35 UTC

1. Conjecture

Field A (mathematical branch): Order theory / Dilworth's theorem on the poset of minimal 0/1-certificates of a Boolean function ordered by sub-assignment, with antichain-width w(f) computed via Koenig-style maximum matching on the certificate bipartite graph (rarely deployed in lifting-theorem lower bounds; distinct from rank/Mobius approaches) **Field B** (complexity object): Deterministic two-party communication complexity D^cc(f o IND_b) of Indexing-gadget-lifted Boolean functions, accessed through deterministic decision-tree depth Q^dt(f) via the Raz-McKenzie / Goos-Pitassi-Watson lifting theorem

Statement:

> For every Boolean function f:{0,1}^n -> {0,1} with n <= 12, let w(f) be the Dilworth antichain-width of the poset P(f) whose elements are the minimal 0-certificates and minimal 1-certificates of f, ordered by sub-assignment refinement (a <= b iff dom(a) subset of dom(b) and a agrees with b on dom(a)). Then Q^dt(f) >= ceil(log_2(w(f) + 1)), and consequently the IND_b-lifted deterministic communication complexity satisfies D^cc(f o IND_b) >= ceil(log_2(w(f)+1)) * log_2(b) - O(log n). Falsifier: a single f with w(f) >= $2^{(Q}$dt(f)+1).

Rationale (proposer's reasoning):

> Lifting theorems convert decision-tree depth to communication depth, but the depth side is usually bounded via adversary or rank arguments; Dilworth width on the certificate poset captures a combinatorial 'spread' of incompatible partial proofs that any decision tree must distinguish, giving a poset-theoretic depth bound that should compose cleanly under IND-lifting. Because antichains of minimal certificates are pairwise incompatible, every deterministic tree must reach distinct leaves on each, forcing log_2(w) depth without invoking algebrization-sensitive algebra. If true, the bound provides a purely order-theoretic certificate that survives the gadget product, opening a non-rank route to monotone-style depth separations.

Taxonomy category: `LIFTING` (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `126edd0186288214`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> Across >=300 Boolean functions (exhaustive canonical n in {3,4,5} plus 200 random n in {6..12} at biases p in {1/4,1/2,3/4}), for every f compute w(f) via Koenig matching on P(f) and Q^dt(f) via memoized minimax; conjecture is SUPPORTED iff Q^dt(f) >= ceil(log2(w(f)+1)) on 100% of instances and the n in {4,5} IND_2 lift satisfies D^cc(f o IND_2) >= ceil(log2(w(f)+1)).

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	0.90	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.78	SAFE	SAFE
ALGEBRIZATION	SAFE	0.88	SAFE	SAFE
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: `NOVEL` against 8 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `Raz-McKenzie lifting theorem indexing gadget certificate complexity decision tree` - `Dilworth antichain certificate poset Boolean function decision tree lower bound` - `minimal certificates bipartite matching deterministic query complexity lifting communication`

Top relevant hits considered: - [http://arxiv.org/abs/2512.11833v1] Soft Decision Tree classifier: explainable and extendable PyTorch implementation - [http://arxiv.org/abs/2211.17214v1] Lifting to Parity Decision Trees Via Stifling - [http://arxiv.org/abs/1302.4207v1] A composition theorem for decision tree complexity - [http://arxiv.org/abs/1306.1593v2] The root posets and their rich antichains - [http://arxiv.org/abs/2511.07739v3] A Lower Bound for the Fourier Entropy of Boolean Functions on the Biased Hypercube - [http://arxiv.org/abs/2208.02526v1] Nearly Optimal Communication and Query Complexity of Bipartite Matching - [http://arxiv.org/abs/1512.01210v1] Nearly optimal separations between communication (or query) complexity and partitions - [http://arxiv.org/abs/2110.11439v3] (Optimal) Online Bipartite Matching with Degree Information

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.4s

5.1 Generated Python source

```python
import random
import math
import json

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_boolean_function(n):
        return [random.choice([0, 1]) for _ in range(2**n)]
```

```
def truth_table_to_dict(tt):
    n = int(math.log2(len(tt)))
    table = {}
    for i in range(len(tt)):
        inputs = tuple((i >> j) & 1 for j in range(n))
        outputs = tt[i]
        if inputs not in table:
            table[inputs] = []
        table[inputs].append(outputs)
    return table

def prime_implicant_enumeration(f):
    n = int(math.log2(len(f)))
    table = truth_table_to_dict(f)
    implicants = []
    for inputs, outputs in table.items():
        if len(set(outputs)) == 1:
            implicants.append(inputs)
    return implicants

def build_poset(implicants):
    poset = {}
    for i in range(len(implicants)):
        for j in range(i + 1, len(implicants)):
            a, b = implicants[i], implicants[j]
            if all(a[k] == b[k] or a[k] == 0 for k in range(len(a))):
                poset.setdefault(a, []).append(b)
    return poset

def maximum_bipartite_matching(graph):
    n = len(graph)
    matching = [-1] * n
    visited = [False] * n

    def dfs(u):
        for v in graph[u]:
            if not visited[v]:
                visited[v] = True
                if matching[v] == -1 or dfs(matching[v]):
                    matching[v] = u
                    return True
        return False

    for u in range(n):
        visited = [False] * n
        dfs(u)

    return sum(1 for x in matching if x != -1) // 2

def memoized_minimax(f, depth=0, cache=None):
    if cache is None:
        cache = {}
    n = int(math.log2(len(f)))
    key = (tuple(f), depth)
    if key in cache:
```

```
            return cache[key]

        if depth == n:
            return 1

        min_val = float('inf')
        for i in range(n):
            left = f[:i] + [0] * (n - i) + f[i+1:]
            right = f[:i] + [1] * (n - i) + f[i+1:]
            min_val = min(min_val, max(memoized_minimax(left, depth + 1, cache), memoized_minimax(

        cache[key] = min_val
        return min_val

    def ind_lift(f, b):
        n = int(math.log2(len(f)))
        protocol_tree = {}
        for i in range(n):
            left = f[:i] + [0] * (n - i) + f[i+1:]
            right = f[:i] + [1] * (n - i) + f[i+1:]
            protocol_tree[(i, 0)] = memoized_minimax(left)
            protocol_tree[(i, 1)] = memoized_minimax(right)

        def dfs(node, depth):
            if node in protocol_tree:
                return protocol_tree[node]
            min_val = float('inf')
            for i in range(n):
                left = f[:i] + [0] * (n - i) + f[i+1:]
                right = f[:i] + [1] * (n - i) + f[i+1:]
                min_val = min(min_val, max(dfs((i, 0), depth + 1), dfs((i, 1), depth + 1)))
            return min_val

        return dfs((0, 0), 0)

    n = random.randint(3, 12)
    f = generate_boolean_function(n)
    implicants = prime_implicant_enumeration(f)
    poset = build_poset(implicants)
    w_f = maximum_bipartite_matching(poset)
    q_dt_f = memoized_minimax(f)

    conjecture_holds = q_dt_f >= math.ceil(math.log2(w_f + 1))
    counterexample = "" if conjecture_holds else f"Counterexample for n={n}, w(f)={w_f}, Q^dt(f)={

    return {
        "metric_name": "Q^dt(f)",
        "metric_value": q_dt_f,
        "instances_tested": 1,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] or [11, 23, 37, 53, 71]
```

```
    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(json.dumps({"TRIAL": {"seed": seed, **result}}))
        results.append(result)

    mean_q_dt_f = sum(r["metric_value"] for r in results) / len(results)
    std_q_dt_f = math.sqrt(sum((r["metric_value"] - mean_q_dt_f) ** 2 for r in results) / len(resu
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_q_dt_f} std={std_q_dt_f} support_fraction={support_fr
    elif sum(1 for r in results if not r["conjecture_holds"]) / len(results) >= 0.2:
        print("RESULT: FALSIFIED counterexample=\"\" first_failing_seed=0")
    else:
        print(f"RESULT: INCONCLUSIVE support_fraction={support_fraction}")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_2595fd2f.py", line 137, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_2595fd2f.py", line 118, in run_trial
    w_f = maximum_bipartite_matching(poset)
          ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_2595fd2f.py", line 69, in maximum_bipar
    dfs(u)
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_2595fd2f.py", line 59, in dfs
    for v in graph[u]:
             ~~~~~^^^
KeyError: 0
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test crashed with a KeyError in the bipartite matching routine before producing any data, so neither the support nor falsification conditions can be evaluated. | next: Fix the maximum_bipartite_matching function to initialize graph[u] as an empty adjacency list (e.g., via defaultdict(list)) and rerun the pre-registered protocol.

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	22121
2	preregistration	claude_max	opus	0	0	7289
3	novelty	claude_max	opus	0	0	3887
4	novelty	claude_max	opus	0	0	8684
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16109
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14911
7	judge	claude_max	opus	0	0	3863

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 76862 ms total latency. Provider mix: {'claude_max': 5, 'ollama_remote': 2}

(full prompt+response transcripts available in `research/audit/52be4733a1c4.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/52be4733a1c4.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/52be4733a1c4.tar.gz` (if generated)